The Last Clue

A 2D detective puzzle game in C++, drawn entirely by our own graphics algorithms

Computer Graphics group project — Final report

Member	GitHub	Role
Caleb Kipchirchir	C-Sawe	Graphics & Algorithms, integration
Alvin Kimani	alvviinn	Environment & Visual Design
Sudheysi Ali	Satasdin	Puzzles & Game Mechanics
Latiffa	latifahjaafar	Interface, Animation & Documentation

Repository: github.com/C-Sawe/the-last-clue · Demo video: docs/demo_video.mp4



The finished game in its OpenGL window: the security panel code puzzle.

1. Overview

A business owner has been found dead in her study. The player is the detective: they explore the study, solve six puzzles, collect six pieces of evidence and finally answer three questions — **who** killed her, **how** and **why**. The right answer (the partner, poison, to hide the money he had stolen) is solved; three wrong accusations close the case unsolved.

The project rule we set ourselves from Checkpoint 1 was that **every pixel is drawn by our own code**. A *Canvas* (a plain pixel buffer) is filled by our implementations of the classic algorithms: Bresenham lines, midpoint circles and ellipses, scanline polygon fill, clipping, Bézier and Catmull-Rom curves, 3×3 transformation matrices and inverse-mapped raster transforms. From Checkpoint 3 the finished Canvas is uploaded to OpenGL as a texture each frame; OpenGL also draws the hover outlines with its own line primitive, and GLFW provides the window, mouse and keyboard. The same game code runs without a window, which is how the scripted playthrough, the screenshots and the demo video are produced.

Three of the puzzles are **code puzzles**: the player edits a short program in ClueScript, our own small language with a lexer, recursive-descent parser and tree-walking interpreter. The program draws with our algorithms, and the game checks the drawing (or the printed text) to decide whether the lock opens.

2. Building and playing

Requirements: CMake 3.14+ and a C++17 compiler. `cmake -B build -S . && cmake --build build`, then run `./build/the_last_clue`. GLFW is used if installed, otherwise CMake downloads it. `-DLASTCLUE_GAME=OFF` builds only the image demos. All 38 demos are run from the repository root and write BMP/GIF files to `output/`; `scripts/render_all.sh` builds and runs them all.

Puzzle	Opens / gives	Graphics behind it
1. Torn letter	“The code is in the painting”	Drag and rotate pieces (transform matrices), picking with point-in-polygon, snapping by position + angle tolerance
2. Safe alarm (code B)	Alarm off; MAIL.CLUE unlocks	ClueScript turtle star (rotation of the heading vector), rasterised and compared by IoU ≥ 0.85
3. Safe dial 37-12-85	Door key + fingerprint	Dial follows the mouse angle (atan2); the door swings open (eased rotation)
4. Security panel (code A)	Powers the study door	Wires traced from terminals, point-in-polygon on sockets, segment-intersection test for crossings
5. Encrypted email (code C)	The motive	Caesar shift implemented as a ClueScript command; the printed text is compared
6. Fingerprints	Who touched the safe	Ellipse-ridge prints; the lifted print is turned by inverse mapping and matched by IoU of ink masks

3. Architecture

Layer	Folder	Contents
Core	src/core	Canvas, colours, BMP writer, GIF encoder (palette + LZW), 5×7 bitmap font, key/button types
Algorithms	src/primitives, polygon, curves, transform	Lines, circles, ellipses, fills, clipping, point-in-polygon, Bézier, Catmull-Rom, 3×3 matrices
Animation	src/animation	lerp, easing (Quad, Cubic, Back, Bounce), keyframe tracks, transitions (cross-fade, fade through black, wipe, iris)
Scenes	src/scenes	The study, characters, safe, torn letter, evidence board, fingerprints, security panel
Language	src/script	ClueScript lexer, parser, AST, interpreter (step limit) and drawing bindings
Game	src/game	Game state machine, picking/dragging, puzzles, code checks, deduction rules, explore screen, the Game loop
Interface	src/ui	Text layout, buttons, code terminal, Case File, evidence board, deduction screen, outcome sequences

Layer	Folder	Contents
Platform	src/platform, apps	GLFW window, fixed-timestep loop, Canvas → OpenGL texture; the only code that uses GLFW/OpenGL

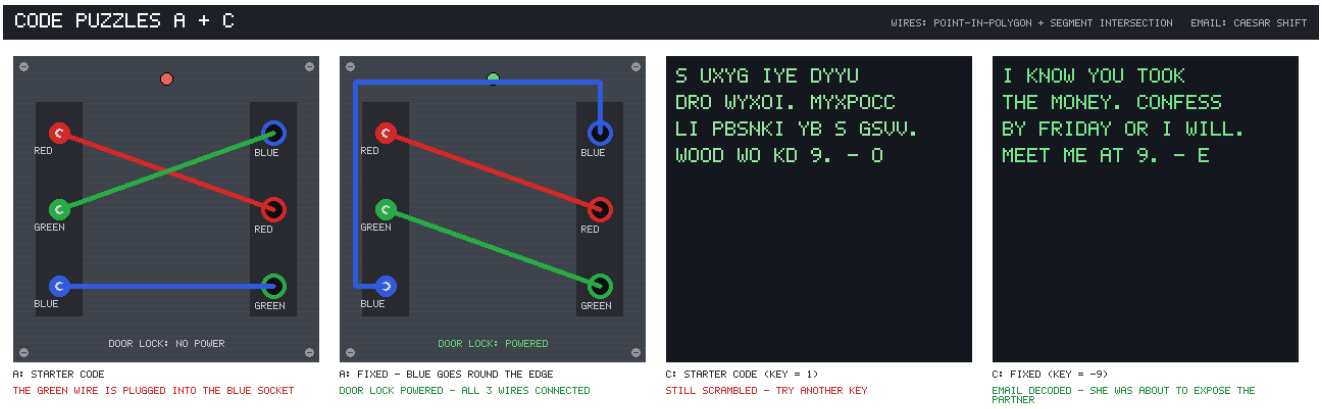
Each frame: input events (already converted to canvas coordinates) go to the Game, which routes them to the active screen; *update(dt)* advances animations and checks which puzzles were solved (setting flags, adding evidence, unlocking the next area); *render* draws the Case File panel and the active screen into the Canvas, blends screen changes with a transition, and the platform layer uploads the result to OpenGL.

4. Graphics techniques and where they are used

Technique	Algorithm	Used for	Code
Lines	DDA, Bresenham (8 octants), thick lines	Strings, wires, outlines, ClueScript lines	primitives/line.cpp
Circles / ellipses	Midpoint, 8-way / 4-way symmetry, filled spans	Dial, pins, heads, rug, fingerprint ridges	primitives/circle.cpp, ellipse.cpp
Polygon fill	Scanline with edge table + active edge table	Every filled shape	polygon/fill.cpp
Seed fill	Flood and boundary fill (explicit stack)	Region colouring	polygon/fill.cpp
Clipping	Cohen–Sutherland, Sutherland–Hodgman	Viewport, magnifier, ClueScript output clipped to the monitor	polygon/clip.cpp, script/render.cpp
Hit testing	Point in polygon (ray casting)	Hotspots, letter pieces, sockets	polygon/clip.cpp, game/input.cpp
Curves	de Casteljau Bézier, Catmull-Rom	Sagging strings, handwriting	curves/
Transforms	3×3 homogeneous: translate, rotate/scale about a pivot, shear, reflection, inverse	Dial, pieces, door, drawers, card flip, stamp	transform/transform2d.cpp
Raster transforms	Inverse mapping through inverse(M)	Painting swing, sprite scaling, magnifier, rotated prints, stamp	scenes/, ui/deduction_ui.cpp
Hierarchical transforms	Limbs rotate about joints	Detective walk cycle	scenes/characters.cpp
Animation	lerp, easing, keyframes, staggered phases	Every movement; card drops, evidence pop, toast slide	animation/
Transitions	Cross-fade, fade through black, wipe, iris	Screen changes (wipe into the deduction, black before the verdict)	animation/transitions.cpp
GIF encoding	Palette + LZW	Every animation in the docs	core/gif.cpp
OpenGL display	Texture upload, orthographic quad, GL_LINE_LOOP	The game window and hover outlines	platform/gl_canvas.cpp
Interpreter	Lexer → recursive descent → AST → evaluator	ClueScript code puzzles	script/
IoU checking	Rasterise with thick lines; $\|A \cap B\| / \|A \cup B\|$	Star puzzle, fingerprint matching	game/code_check.cpp, puzzles.cpp
Segment intersection	Orientation (cross product) test + collinear cases	Wiring: no wires may cross	game/code_check.cpp
Vanishing-point scaling	Scale about a horizon point	The culprit walking free (CASE FAILED)	ui/deduction_ui.cpp

5. The final-build puzzles

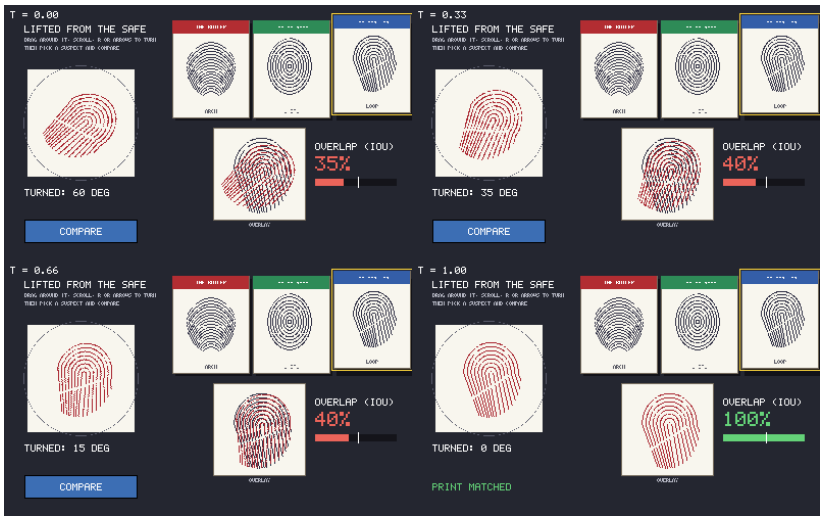
Wiring (code A). The panel has three coloured terminals and three sockets in a different order. The player's program draws lines; the checker groups them into wires by colour, follows each wire from its terminal (a segment joins when one of its ends is within 8 px of a point the wire already reaches), tests the wire's end points against each socket with point-in-polygon, and finally runs the segment-intersection test on every pair of segments from different wires. A straight blue wire would cross the other two, so it has to be routed round the edge — which a single `line` per wire cannot do.



Code puzzles A and C: the starter code fails with a specific message; the fixed code passes.

Email (code C). The victim's last email was Caesar-shifted by 9 (the clock stopped at 9). ClueScript gained a `shift "TEXT" n` command (modular arithmetic on letters); the program must print the decoded message. Looping over all 26 keys also passes, because the check looks for the message anywhere in the output.

Fingerprints. Each suspect's print is built from concentric midpoint ellipses shaped into a whorl, a loop or an arch, cut to a finger outline with a point-in-ellipse test. The print lifted from the safe starts turned 60°; the player turns it (in 5° steps) and picks a suspect. The score is the IoU of the two prints' ink masks after inverse-mapped rotation: the partner scores 100% when lined up and about 55% at 5° off, while the other suspects never exceed 50% at any angle, so the pass mark is 53%.



Turning the lifted print back from 60° to 0°: the overlap with the partner rises to 100%.

6. Interface and animation

The code terminal now holds several files as tabs (the email tab stays locked until the alarm is off); a puzzle can supply a background (the wiring board) or show printed text instead of a drawing, and a hint appears after three failed runs. The Case File has six evidence slots edged in the evidence category colour; a new piece pops into its slot with an OutBack scale about the slot centre. Messages slide up and down instead of popping. The deduction cards drop in one after another and lift when chosen. The verdict is a rubber stamp rendered into its own canvas, then scaled from 3× to 1× with an overshoot and rotated by inverse mapping, followed by a short shake; the story types itself out and the partner's mug shot slides in. On a lost case he walks away, scaled down about a vanishing point.

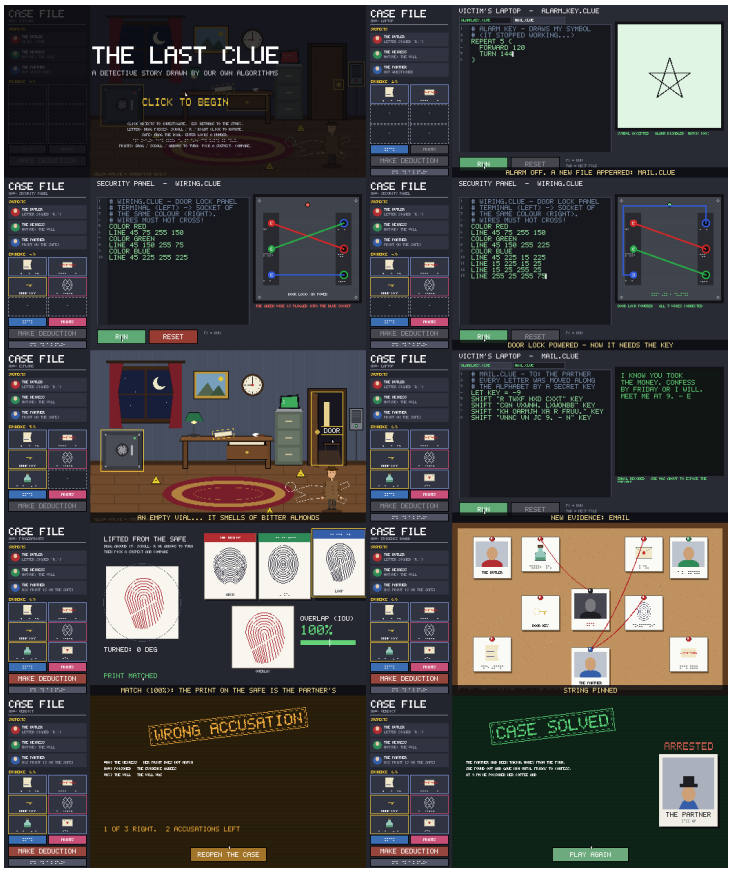


The three outcomes: CASE SOLVED, WRONG ACCUSATION (two tries left), CASE FAILED.

7. Testing

Scripted playthrough. `38_final_playthrough` drives *game::Game* with the same mouse and keyboard events a player produces — it walks to objects, drags letter pieces, turns the dial, types code into the terminals, turns the print, ties strings and accuses. It finishes with all six pieces of evidence, all puzzle flags set, one wrong accusation and CASE SOLVED. With `LASTCLUE_LOSE=1` it makes three wrong accusations instead and checks that the third closes the case and that PLAY AGAIN starts a fresh case. With `LASTCLUE_VIDEO` it records the demo video through ffmpeg.

Real window. The OpenGL build was run under a virtual X display with simulated clicks (title → security panel → RUN → door), and its screenshots are in `docs/screenshots/gl_final_*.png`. **Regression.** Every demo (01–38) builds with no compiler warnings (-Wall -Wextra) and runs; images that changed since earlier checkpoints were reviewed and regenerated.



Key frames of the scripted playthrough, from the title to CASE SOLVED.

8. Contributions

Member	Final project
Caleb	Final Game integration: six puzzles wired together, unlock rules, outcome screens, play again, transitions and sliding toast; scripted playthrough with video recording and lose-path test; bug bash and release

Member	Final project
Alvin	Fingerprint art (ellipse ridges), security panel and wiring board, back room and vial, alarm and panel LEDs, shared feedback/evidence palette; new hotspots and the two-lock door
Sudheysi	Code puzzles A (wiring) and C (email) with starter code and hints, the Caesar-shift command, fingerprint matching logic, deduction rules and the three-tries limit
Latiffa	Multi-file code terminal, Case File with six slots and evidence pop, deduction screen, CASE SOLVED / FAILED sequences; README, this report and the demo video

Earlier checkpoints are documented in docs/checkpoint1_screenshots.pdf, checkpoint2_animations.pdf and checkpoint3_prototype.pdf.

9. Limitations and future work

Drawing is done on the CPU, so a larger window scales the frame rather than drawing more detail; nothing is anti-aliased and our font is upper-case only. ClueScript has no functions or lists and the editor has no selection or copy/paste. There is one fixed case and no save game. Natural next steps are a second case built from the same components, anti-aliased lines (Xiaolin Wu), and GPU drawing of the primitives now that the OpenGL path exists.